

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



New Trend in ICT (CO5179)

Mini-Project

Designing a Lambda Architecture for Real-Time IoT Data in a
Vietnamese Smart City Scenario

Lecturer:	Thoai Nam	
Students:	Krebs Luca Felix	2470475
	Onuh Izuchukwu Justus	2470477
	Nguyễn Ngọc Minh Xuân	2470473

HO CHI MINH CITY, NOVEMBER 2025



Table of Contents

List of Figures	3
List of Tables	4
1 Introduction	6
1.1 Background and Motivation	6
1.2 Problem Statement	6
1.3 Objectives and Research Questions	7
1.4 Scope and Contributions	8
2 Related Work and Theoretical Background	9
2.1 Smart Cities and Smart Villages	9
2.2 IoT Data and Urban Sensing	9
2.3 Big Data Architectures and Lambda Architecture	9
2.4 Edge, Fog, and Cloud Computing	10
3 Use Case and Data Sources	12
3.1 Use Case: Low-Emission Traffic Management in Ho Chi Minh City	12
3.2 Batch Layer Data: HCMC Traffic Flow Dataset	12
3.3 Speed Layer Data: Real-Time Air Quality for HCMC	13
3.4 Data Preprocessing and Assumptions	14
4 System Design: Lambda Architecture	15
4.1 Overall Architecture Overview	15
4.2 Batch Layer Design	16
4.3 Speed Layer Design	17
4.4 Serving Layer and User-Facing Views	17
4.5 Technology Stack (Conceptual)	18
5 Implementation	19
5.1 Development Environment	19
5.2 Batch Processing Pipeline	19
5.3 Streaming / Polling Pipeline	20
5.4 Integration and Data Storage	20
6 Evaluation	22
6.1 Evaluation Criteria	22
6.2 Experimental Setup	22
6.3 Results	23
6.4 Discussion	23



7	Conclusion and Future Work	25
7.1	Summary of Findings	25
7.2	Limitations	25
7.3	Future Work	27



List of Figures

- 1 Architectural variants of the Lambda Architecture prototype: (a) Kafka-based streaming, (b) lakehouse storing only processed views, and (c) lakehouse storing both raw data and processed views. . . . 16



List of Tables

1	Edge, Fog, and Cloud Computing Layers	10
2	Critical Period Identification Thresholds	12
3	Data Sources Comparison	13
4	Comparison of Lambda Architecture Layers	15
5	Technology Stack	19
6	Evaluation Criteria	22
7	Prototype Limitations	26



Abstract

This project designs and implements a small Lambda Architecture that combines historical traffic data from Ho Chi Minh City with near real-time air quality measurements for the same area. In the batch layer, we aggregate a public Kaggle traffic-flow dataset to capture long-term congestion patterns, while in the speed layer, we continuously collect current PM2.5 values from an open air-quality API. The serving layer then integrates both views to identify critical periods with simultaneously high traffic and poor air quality, and we qualitatively evaluate the architecture in terms of latency, scalability, and practical usefulness for Smart City decision-making.

1 Introduction

1.1 Background and Motivation

The rapid urbanization of the 21st century has given rise to Smart Cities, representing a paradigm shift towards data-driven, technology-enabled urban management. Smart Cities leverage information and communication technologies (ICT) to enhance urban services, optimize resource utilization, and improve quality of life through the integration of Internet of Things (IoT) sensors, data analytics platforms, and decision support systems. As over 68% of the world's population is projected to reside in urban areas by 2050 [1], the imperative for intelligent urban management has never been more critical.

Ho Chi Minh City (HCMC), Vietnam's largest metropolitan area with over 9 million residents, exemplifies the challenges of rapid urbanization in developing economies. The city has experienced unprecedented growth over the past three decades, transforming into a sprawling megacity where infrastructure faces significant strain, particularly in transportation and environmental management. Traffic congestion has become a daily reality, with average vehicle speeds during peak hours often dropping below 20 kilometers per hour, while air quality concerns have gained prominence as vehicular emissions have increased substantially.

The intersection of traffic management and environmental monitoring represents one of the most pressing challenges for HCMC. Traffic congestion not only imposes economic costs but also contributes significantly to air pollution, creating a compounding negative impact on public health. Particulate matter (PM2.5) has been identified as a major health concern, with HCMC frequently experiencing levels exceeding World Health Organization guidelines [2], particularly during periods of high traffic density. Understanding the relationships between traffic patterns and air quality enables city administrators to develop targeted interventions and identify critical periods where both conditions converge, supporting public health advisories and evidence-based policy development.

1.2 Problem Statement

Traditional data analytics architectures in urban management follow two distinct paradigms, each with fundamental limitations. Batch processing systems excel at historical analysis but suffer from inherent latency, making them unsuitable for time-sensitive applications. Conversely, real-time analytics systems provide immediate responses but lack historical context and cannot leverage long-term patterns. Additionally, traffic and air quality data are typically isolated in separate analytical silos, preventing city administrators from understanding complex interactions between transportation patterns and environmental conditions.

The Lambda Architecture, originally proposed by Nathan Marz [3], addresses these limitations by providing a unified framework that simultaneously supports both batch and real-time processing. The architecture maintains an immutable master dataset with two parallel processing paths: a batch layer that periodically recomputes comprehensive historical views, and a speed layer that processes recent data in near real-time. The serving layer merges these complementary views, enabling queries to access both comprehensive historical context and current conditions [3]. For HCMC, this dual-path approach enables identification of critical periods where historical traffic patterns indicate likely congestion while real-time air quality measurements confirm elevated pollution levels, creating actionable intelligence for both immediate interventions and long-term planning.

1.3 Objectives and Research Questions

The primary objective of this research is to design, implement, and evaluate a Lambda Architecture prototype that integrates historical traffic flow data with real-time air quality measurements for Ho Chi Minh City. This encompasses developing a conceptual architecture mapping the three-layer structure to urban monitoring requirements, implementing a functional prototype using Python, PySpark, and distributed storage systems, integrating real-world data sources, and qualitatively evaluating the architecture's effectiveness in identifying critical periods where traffic congestion and air pollution coincide.

This research is guided by four primary research questions:

- RQ1 asks: *How can historical traffic data and real-time air quality data be integrated in a single Lambda Architecture?* This addresses the technical challenge of combining datasets with different temporal characteristics within a unified framework.
- RQ2 asks: *What are the trade-offs of this Lambda Architecture in terms of latency, complexity, and data quality?* This provides a comprehensive assessment of costs and benefits.
- RQ3 asks: *How effectively can combined batch and speed views identify critical periods of high traffic congestion and poor air quality?* This examines practical utility and actionable insights.
- RQ4 asks: *What are the scalability and deployment considerations for extending this prototype to a production Smart City platform?* This addresses long-term viability and production requirements.

1.4 Scope and Contributions

The scope of this research is constrained to a conceptual design and lightweight prototype implementation, focusing on District 10 of Ho Chi Minh City as a representative urban area. The prototype utilizes Python and PySpark, demonstrating integration with multiple storage approaches including JSON files, Apache Kafka, and Delta Lake on MinIO. The research excludes full-scale distributed deployments, comprehensive production features, user interface development, and extensive performance benchmarking, maintaining focus on demonstrating core architectural concepts.

The contributions encompass four primary areas. First, the development of a conceptual Lambda Architecture tailored to HCMC, demonstrating how the three-layer structure maps to urban data processing requirements. Second, the implementation of functional batch and speed processing layers operating on real-world datasets, validating practical feasibility. Third, a qualitative evaluation of benefits and limitations, providing insights into trade-offs between complexity and analytical capabilities. Fourth, the demonstration of multiple storage approaches (JSON, Kafka, Lakehouse) within a single framework, showing flexibility in technology choices while maintaining core architectural principles.

2 Related Work and Theoretical Background

2.1 Smart Cities and Smart Villages

Smart Cities represent a comprehensive framework for addressing 21st-century urbanization challenges through strategic integration of information and communication technologies. Giffinger et al. [4] proposed a widely cited framework identifying six dimensions: Smart Economy, Smart People, Smart Governance, Smart Mobility, Smart Environment, and Smart Living. The evolution has progressed from technology-driven approaches to citizen-centric models emphasizing holistic integration of technology, infrastructure, and social systems.

Application domains include intelligent transportation systems, environmental monitoring, public safety, e-government, energy management, and healthcare. These domains share common characteristics: they generate large data volumes, require both real-time monitoring and historical analysis, and benefit from cross-system integration. The concept of Society 5.0 contextualizes Smart Cities within human-centric digital transformation, emphasizing that technology must serve human needs and values.

2.2 IoT Data and Urban Sensing

IoT data in urban environments exhibits distinctive characteristics conceptualized through the four Vs framework: Volume, referring to enormous data generation from thousands of sensors; Velocity, characterizing high-frequency continuous data streams requiring near real-time processing; Variety, encompassing heterogeneous data sources, formats, and structures; and Veracity, addressing data quality challenges from sensor malfunctions, communication failures, and environmental interference.

Urban sensing technologies include traffic sensors (loop detectors, cameras, GPS), environmental sensors (air quality, water quality, weather stations), and infrastructure sensors (structural health monitoring, smart meters). These systems face challenges including deployment costs, maintenance logistics, communication infrastructure requirements, power supply constraints, data privacy concerns, and interoperability issues across diverse vendors and protocols.

2.3 Big Data Architectures and Lambda Architecture

Big Data is characterized by the four Vs: Volume, Velocity, Variety, and Veracity, requiring specialized architectures beyond traditional database systems. Batch processing architectures (e.g., Hadoop [5]) excel at comprehensive historical analysis but suffer from inherent latency. Stream processing architectures (e.g., Apache

Flink [6]) prioritize low-latency real-time processing but lack historical context. Hybrid architectures attempt to combine both but create operational complexity. The Lambda Architecture, proposed by Nathan Marz [3], provides a unified framework eliminating the forced choice between batch and stream processing. It consists of three layers: the batch layer maintains an immutable master dataset and periodically recomputes comprehensive views; the speed layer processes recent data in near real-time for low-latency views; and the serving layer merges both views, providing unified access to historical and real-time data [3]. The architecture's design principles ensure data integrity through immutability, enable scalability through distributed processing, and provide fault tolerance through recomputation [3]. These characteristics align particularly well with IoT and Smart City use cases, where both historical pattern analysis and real-time monitoring are essential.

2.4 Edge, Fog, and Cloud Computing

The distributed nature of Smart City IoT systems has given rise to hierarchical edge-fog-cloud computing architectures that distribute processing, storage, and analytics across multiple layers. The characteristics of each layer are compared in Table 1.

Table 1: Edge, Fog, and Cloud Computing Layers

Characteristic	Edge Layer	Fog Layer	Cloud Layer
Location	Network periphery	Intermediate nodes	Centralized infrastructure
Processing	Minimal operations	Substantial aggregation	Comprehensive preprocessing
Storage	Minimal buffering	Hours to days	Years to decades
Function	Data collection	Local analytics	Advanced analytics
Latency	Lowest	Medium	Highest

The edge layer represents sensors and embedded devices at the network periphery, performing minimal processing and data collection. The fog layer consists of intermediate computing nodes that aggregate data from multiple edge devices, perform preprocessing and local analytics, and manage data transmission to cloud infrastructure. The cloud layer provides centralized computing infrastructure with extensive computational resources, massive storage capacity, and sophisticated analytics capabilities.

Data flow follows a hierarchical path: edge devices collect and validate data, fog nodes aggregate and preprocess data streams, and the cloud layer integrates data from diverse sources for comprehensive analytics and long-term storage. Each



layer performs appropriate preprocessing and aggregation functions, with edge devices performing minimal operations, fog nodes performing substantial aggregation to reduce data volume, and cloud infrastructure performing comprehensive preprocessing for advanced analytics. Storage follows a hierarchical retention strategy, with edge devices maintaining minimal buffering, fog nodes maintaining intermediate-term storage for hours to days, and cloud infrastructure maintaining comprehensive long-term archives spanning years or decades.

3 Use Case and Data Sources

3.1 Use Case: Low-Emission Traffic Management in Ho Chi Minh City

This use case addresses the intersection of traffic congestion and air pollution in HCMC, focusing on identifying critical time periods when both conditions reach elevated levels simultaneously. The use case centers on District 10, a representative urban district with high population density, significant traffic volumes, and mixed land use. The primary goal is to detect periods where average vehicle speeds fall below 20 km/h (indicating severe congestion) while PM2.5 concentrations exceed $50 \mu\text{g}/\text{m}^3$ (indicating unhealthy air quality), requiring temporal alignment through hourly aggregation windows. The critical period thresholds are summarized in Table 2.

Table 2: Critical Period Identification Thresholds

Condition	Threshold
Traffic Congestion	Average vehicle speed $< 20 \text{ km/h}$
Air Pollution	PM2.5 concentration $> 50 \mu\text{g}/\text{m}^3$
Temporal Alignment	Hourly aggregation windows

Stakeholders include city decision-makers requiring both real-time operational data and historical analysis, citizens needing accessible information for travel decisions, and researchers analyzing transportation-environment relationships. Expected outcomes include dynamic traffic management policies, public health advisories, infrastructure planning insights, and research applications. This scenario is ideal for demonstrating Lambda Architecture principles as it addresses genuine urban challenges, leverages authentic data sources, aligns with the architecture's dual-path design, and demonstrates clear business value for both operational and strategic decision-making.

3.2 Batch Layer Data: HCMC Traffic Flow Dataset

The batch layer processes historical traffic flow data from the TomTom Traffic API [7] (Flow Segment Data API v4), which provides traffic information for road segments worldwide including HCMC. The speed layer processes real-time air quality measurements from the OpenAQ API [8], a global open-source platform aggregating air quality data from multiple sources. A comparison of both data sources is provided in Table 3.

Table 3: Data Sources Comparison

Characteristic	TomTom Traffic API	OpenAQ API
Data Type	Historical traffic flow	Real-time air quality
Layer Assignment	Batch layer	Speed layer
Update Frequency	Historical snapshots	5-minute polling
Key Attributes	Speed, coordinates, timestamps	PM2.5, station ID, coordinates
Coverage	District 10 grid points	3km radius, up to 10 stations
Data Format	JSON, ISO 8601 timestamps	JSON, UTC timestamps
Retention	Long-term historical	24-48 hours

The traffic data schema includes timestamps in ISO 8601 format (Asia/Ho_Chi_Minh, UTC+7), geographic coordinates (latitude, longitude), current speed and free flow speed in km/h, and optional metadata including road segment identifiers and confidence scores. The batch layer performs hourly aggregation, calculating average speeds, minimum and maximum speeds, speed standard deviations, and measurement counts for each hour and geographic region. Data volumes vary with collection period and spatial coverage, with typical implementations generating substantial datasets requiring distributed processing. The comprehensive temporal coverage enables identification of recurring congestion patterns, long-term trends, seasonal variations, and the impact of infrastructure changes, supporting both strategic planning and operational decision-making.

3.3 Speed Layer Data: Real-Time Air Quality for HCMC

The speed layer processes real-time air quality measurements from the OpenAQ API, a global open-source platform aggregating air quality data from multiple sources. The API requires authentication through an API key and supports multiple parameters, though this implementation focuses on PM2.5 measurements due to their relevance for public health and correlation with traffic-related pollution. The prototype queries stations within a 3-kilometer radius of District 10's center coordinates, querying up to 10 monitoring locations to provide comprehensive coverage.

The polling frequency is configurable, defaulting to 5 minutes, providing a balance between data freshness and API usage. The data extraction process retrieves timestamps (converted from UTC to local time), station identifiers, PM2.5 values in $\mu\text{g}/\text{m}^3$, geographic coordinates, and optional data quality indicators. The speed

layer's data retention policy maintains only recent data (typically 24-48 hours), discarding older measurements once superseded by batch-processed views, ensuring manageable storage while preserving freshness for real-time applications.

3.4 Data Preprocessing and Assumptions

Data preprocessing transforms raw sensor data and API responses into clean, consistent formats using Python libraries (Pandas [9], NumPy [10]) and custom utility functions. The preprocessing addresses missing values through validation checks, forward-fill/backward-fill methods for short gaps, and aggregation strategies. Outlier detection employs statistical methods (z-score, IQR) and domain-specific validation, distinguishing between errors (removed) and genuine extreme events (preserved). Data cleaning includes format standardization, unit conversion, and data type validation, with timestamp parsing and timezone normalization to Asia/Ho_Chi_Minh (UTC+7).

Time alignment employs hourly aggregation windows aligned to standard hour boundaries, enabling comparison and integration despite different measurement frequencies. The alignment handles edge cases including timezone differences and data collection gaps. Spatial assumptions treat traffic measurements at grid points and air quality measurements from monitoring stations as representative of district-wide conditions, enabling temporal alignment without precise spatial matching. Data quality assumptions acknowledge that API sources provide reasonably accurate measurements, with validation checks and error handling managing temporary unavailability, while accepting that some missing data is expected and can be handled gracefully.

4 System Design: Lambda Architecture

4.1 Overall Architecture Overview

The Lambda Architecture implementation for HCMC Smart City traffic and air quality integration follows a hierarchical data flow from data collection through processing layers to comprehensive analytical views [3]. The architecture consists of three main layers, whose characteristics are compared in Table 4.

Table 4: Comparison of Lambda Architecture Layers

Characteristic	Batch Layer	Speed Layer	Serving Layer
Data Scope	All historical data	Recent data (24-48h)	Combined views
Processing Mode	Periodic recomputation	Continuous incremental	On-demand queries
Latency	Hours to days	Minutes	Seconds to minutes
Data Source	TomTom Traffic API	OpenAQ API	Batch + Speed views
Output Format	CSV, Parquet, Delta Lake	Append-only CSV/Delta	Integrated tables
Primary Function	Historical patterns	Real-time conditions	Unified access

Data collection occurs from the TomTom Traffic API [7] and OpenAQ API [8], with multiple storage modes including JSON files, Apache Kafka [11] topics, or Delta Lake [12] tables. The batch layer performs temporal and spatial aggregation to create hourly summaries, generating precomputed views stored as CSV, Parquet, or Delta Lake tables. The speed layer continuously polls the OpenAQ API, maintaining append-only views of recent PM2.5 measurements. The serving layer performs temporal alignment and identifies critical periods where both traffic congestion and air pollution occur simultaneously. The architecture supports multiple storage backends and utilizes PySpark for distributed processing, while maintaining compatibility with single-machine prototype deployments.

The prototype demonstrates three architectural variants, each optimized for different storage and processing requirements, as shown in Figure 1. Figure a illustrates the Kafka-based architecture, where raw data from both APIs is ingested into Kafka topics (`traffic-raw` and `air-quality-raw`), enabling real-time streaming capabilities for both batch and speed layers. Figure b presents a lakehouse architecture variant without raw data storage, where processed views are directly stored in S3 (MinIO [13]) as Delta Lake tables. Figure c shows an extended lakehouse architecture that includes raw data storage alongside processed views, maintaining both raw data archives and processed views in Delta Lake tables and thus preserving the immutable master dataset principle [3].

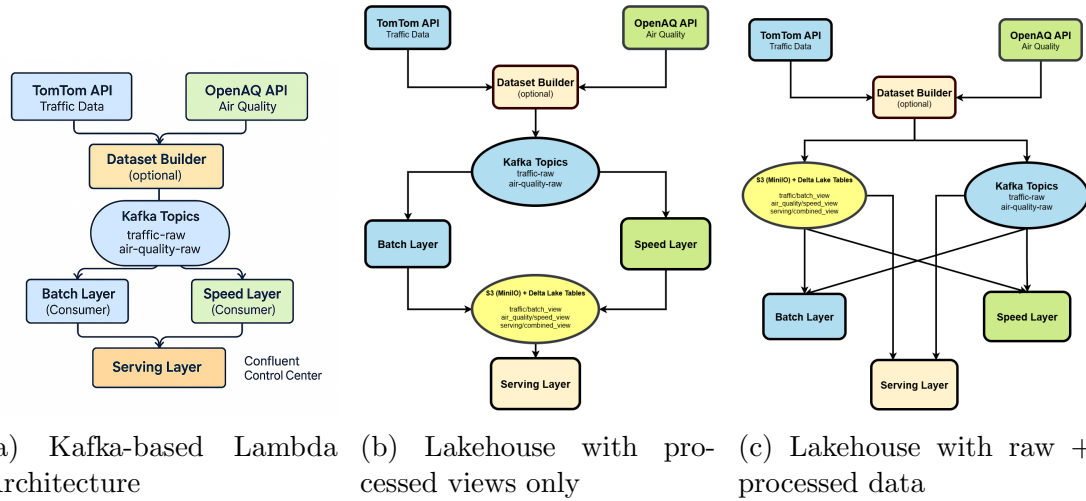


Figure 1: Architectural variants of the Lambda Architecture prototype: (a) Kafka-based streaming, (b) lakehouse storing only processed views, and (c) lakehouse storing both raw data and processed views.

4.2 Batch Layer Design

The batch layer processes the entire immutable master dataset following the Lambda Architecture principle of recomputation [3], where comprehensive views are periodically recomputed over all historical data. Input data includes historical traffic flow data from the TomTom Traffic API, stored in formats including JSON files, Kafka topics, or Delta Lake tables, with attributes including timestamps, geographic coordinates, current and free flow speeds, and optional metadata.

Processing logic implements periodic offline jobs (typically daily or hourly) that perform temporal aggregation into hourly time windows, calculating aggregate statistics including mean, minimum, maximum speeds, standard deviations, and measurement counts. Spatial aggregation complements temporal aggregation, grouping measurements by geographic regions. The aggregation operations utilize PySpark [14] for distributed processing, with data validation steps ensuring high-quality batch views.

Output specification defines batch views stored as CSV, Parquet, or Delta Lake tables, partitioned by date or hour for efficient querying. The batch views provide comprehensive historical context for the serving layer, support standalone historical analysis, and serve as the authoritative source for historical traffic data. Performance considerations include processing time scaling with data volume, computational resource requirements, and scalability through distributed processing.

4.3 Speed Layer Design

The speed layer provides low-latency views of recent air quality conditions through incremental processing, operating continuously on recent data (typically 24-48 hours) rather than the entire historical dataset. Input data arrives through API polling of the OpenAQ API at configurable intervals (defaulting to 5 minutes), retrieving PM2.5 measurements with attributes including timestamps, station identifiers, PM2.5 values, and geographic coordinates.

The polling mechanism implements a continuous loop with error handling and retry logic using exponential backoff. Data parsing transforms API responses into standardized formats, converting timestamps to local time and validating PM2.5 values. Filtering selects PM2.5 measurements from stations within District 10's geographic area, with optional sliding window aggregates for trend identification. Output specification defines append-only speed views stored as CSV files or Delta Lake tables, containing recent PM2.5 measurements with timestamps. The speed layer's output serves the serving layer's integration operations, with views periodically superseded by batch-processed views. End-to-end latency ranges from minutes to the polling interval, substantially lower than batch layer latency, demonstrating real-time responsiveness for time-sensitive applications.

4.4 Serving Layer and User-Facing Views

The serving layer combines batch and speed views to provide unified access to both historical and real-time data. The integration strategy employs temporal alignment on hourly time windows, with both sources using identical hourly boundaries to enable efficient joining operations. The join operation combines views on temporal keys, typically using inner joins for critical period identification where both data sources are required.

Critical period identification applies threshold-based filtering: traffic congestion when speeds fall below 20 km/h, and air pollution when PM2.5 exceeds 50 $\mu\text{g}/\text{m}^3$. The algorithm flags time periods meeting both criteria and may calculate additional metrics such as severity scores and duration. Output formats include integrated view tables (CSV or database tables), visualization outputs (time series plots, scatter plots, critical period timelines), and summary statistics (counts, duration metrics, temporal patterns, trend metrics). The prototype focuses on generating data outputs that can be consumed by various interfaces, while user-facing interfaces would be developed as separate components based on specific requirements.

4.5 Technology Stack (Conceptual)

The prototype utilizes Python 3.9+ [15] as the primary programming language, with core data processing libraries including Pandas [9] for data manipulation, NumPy [10] for numerical computing, and PySpark 3.5.0 for distributed processing. API integration uses the requests library and openaq Python SDK, with dateutil and pytz for timezone handling. Visualization employs Matplotlib and Seaborn for publication-quality plots.

Storage solutions demonstrate multiple approaches: JSON/CSV files for simple file-based storage, Apache Kafka for production-like streaming with immutable log semantics [3], and Delta Lake on MinIO for lakehouse architectures. The conceptual mapping to production technologies illustrates how prototype components translate to enterprise-scale deployments: Python processing maps to containerized Kubernetes [16] deployments, PySpark local mode maps to distributed Spark clusters, file-based storage maps to HDFS or cloud storage, and batch/speed processing map to workflow orchestration tools (Airflow [17]) and stream processing frameworks (Flink, Kafka Streams). The prototype's simplified deployment operates on a single machine while maintaining conceptual compatibility with production architectures, demonstrating core principles that can be extended with appropriate infrastructure scaling.

5 Implementation

5.1 Development Environment

The prototype utilizes Python 3.9+ [15] as the primary programming language, chosen for its extensive ecosystem of data processing libraries and widespread adoption in data science applications. Development was conducted on multiple operating systems (Windows, Linux, macOS) with platform-agnostic codebase design. Dependency management uses `requirements.txt` and `pyproject.toml`, with Git for version control and Black code formatter for consistent styling.

The project structure organizes code into logical modules: `batch_layer/`, `speed_layer/`, `serving_layer/`, and `utils/`, enabling independent development and testing. The complete source code and implementation are available in the project repository [18]. The development workflow emphasizes iterative development with configuration management through `config.py`, using environment variables for sensitive information. Core dependencies are categorized in Table 5.

Table 5: Technology Stack

Category	Technologies
Programming Language	Python 3.9+
Data Processing	Pandas, NumPy, PySpark 3.5.0
Storage	Delta Lake 3.1.0, JSON, CSV, Apache Kafka
API Integration	requests, openaq SDK
Visualization	Matplotlib, Seaborn
Timezone Handling	dateutil, pytz
Storage Libraries	s3fs, boto3, kafka-python

5.2 Batch Processing Pipeline

The batch processing pipeline implements the Lambda Architecture's batch layer in the `batch_layer` module, supporting multiple data sources including JSON files, Kafka [11] topics, and Delta Lake [12] tables. The pipeline follows a modular design with separate functions for each processing step.

Data loading reads from configured sources (JSON, Kafka, or Delta Lake) with initial validation. Timestamp parsing handles multiple formats, converting to local time (Asia/Ho_Chi_Minh, UTC+7) and creating temporal keys for hourly aggregation. Data cleaning handles missing values, outliers (using z-score and IQR methods), and invalid measurements, distinguishing between errors and genuine

extreme events. Temporal aggregation groups measurements into hourly windows, calculating mean, min, max speeds, standard deviations, and measurement counts. Spatial aggregation complements temporal aggregation, grouping by geographic regions. Output generation writes batch views as CSV, Parquet, or Delta Lake tables, partitioned by date or hour. The code structure organizes functionality into logical functions with error handling and logging. Performance scales with data volume, with processing times ranging from minutes to hours depending on dataset size and computational resources.

5.3 Streaming / Polling Pipeline

The speed layer pipeline continuously collects and processes real-time air quality measurements, implemented in the `speed_layer` module. API integration utilizes the OpenAQ Python SDK (`openaq_client.py`), handling authentication through environment variables and implementing `fetch_pm25_measurements()` with parameters for geographic coordinates, search radius, time window, and maximum locations.

The scheduling and polling strategy employs a configurable polling interval (defaulting to 5 minutes) through a continuous loop, balancing data freshness with API usage. The polling mechanism includes comprehensive error handling with exponential backoff retry logic (maximum 3 attempts), distinguishing between transient and permanent errors. Data parsing transforms API responses into standardized formats, converting timestamps from UTC to local time and validating PM2.5 values. Data storage utilizes append-only structures (CSV files or Delta Lake tables), enabling incremental updates while maintaining temporal ordering. The data retention policy maintains only recent data (24-48 hours), discarding older measurements once superseded by batch-processed views.

5.4 Integration and Data Storage

The serving layer integrates batch and speed views through the `data_integrator.py` module, which reads precomputed views from both layers and performs temporal alignment on hourly time windows. Batch views are stored in `data/batch_views/` as CSV or Parquet files, while speed views are stored in `data/speed_views/` as append-only CSV files or Delta Lake tables. The integration process loads both views, performs temporal alignment by matching hourly timestamps, and executes join operations (typically inner joins) to combine traffic and air quality data.

The data model for combined views includes columns for timestamp (hourly window), average traffic speed, PM2.5 concentration, and critical period indicators. Helper utilities in `utils/` include `data_loader.py` for common loading functions



and `visualizer.py` for generating time series plots, scatter plots, and critical period timelines. The integrated views are stored in `data/serving_views/` as CSV files with visualization outputs saved to `data/serving_views/plots/`. The storage architecture supports multiple backends: simple file-based storage for development, Kafka topics for production-like streaming, and Delta Lake on MinIO [13] for scalable lakehouse architectures, with the implementation maintaining compatibility across all approaches.

6 Evaluation

6.1 Evaluation Criteria

The evaluation of the Lambda Architecture prototype employs multiple criteria to assess both technical performance and practical utility, as summarized in Table 6.

Table 6: Evaluation Criteria

Criterion	Description
Latency and Data Freshness	Time from data generation to availability in queryable views; batch layer provides historical context (hours to days), speed layer provides near real-time responsiveness (minutes)
Scalability	Conceptual scalability characteristics with larger datasets, more concurrent users, and distributed deployments, acknowledging prototype limitations
Complexity	Implementation complexity (code structure, component count, integration) and operational complexity (deployment, maintenance, monitoring)
Expressiveness	Quality and relevance of identified critical periods and relationships between traffic and air quality patterns for Smart City decision-making

6.2 Experimental Setup

The experimental evaluation utilized data collected over a specific time period, with traffic data spanning several weeks of historical measurements from District 10 and air quality data collected through continuous API polling. The prototype was executed on a standard development machine with typical laptop specifications, running Python 3.9+ with PySpark in local mode. The dataset included thousands of traffic measurements aggregated into hourly batch views, and hundreds of air quality measurements collected through the 5-minute polling interval. The evaluation focused on identifying critical periods where both traffic congestion (speeds below 20 km/h) and air pollution (PM2.5 above 50 $\mu\text{g}/\text{m}^3$) occurred simultaneously, analyzing temporal patterns, correlations, and the effectiveness of the integrated views in supporting decision-making applications.

6.3 Results

The integrated system successfully identified multiple critical periods where traffic congestion and air pollution coincided, demonstrating the practical value of combining batch and speed views. Analysis revealed recognizable temporal patterns, with critical periods occurring more frequently during morning and evening rush hours (7-9 AM and 5-7 PM), aligning with expected traffic congestion patterns. The correlation analysis between traffic speeds and PM2.5 concentrations showed expected relationships, with lower average speeds generally corresponding to higher pollution levels, though the relationship exhibited variability due to factors such as weather conditions and non-traffic pollution sources.

Visualization outputs including time series plots and scatter plots effectively illustrated the relationships between traffic and air quality, making complex data patterns accessible for analysis. The critical period timelines highlighted specific time windows requiring attention, enabling city administrators to identify high-risk periods for coordinated interventions. The Lambda Architecture's dual-path design successfully provided both comprehensive historical context from batch views and current conditions from speed views, demonstrating that the integration of both perspectives yields insights that would not be available through either data source independently.

6.4 Discussion

The evaluation results demonstrate that the Lambda Architecture successfully integrates historical traffic data and real-time air quality measurements, providing actionable insights for Smart City decision-making. The identification of critical periods enables both immediate operational responses and long-term strategic planning, with batch views revealing recurring patterns and speed views providing current conditions. The architecture's expressiveness is evident in its ability to answer questions that require both historical analysis and real-time monitoring, such as identifying whether current conditions represent typical patterns or unusual events.

However, the evaluation also reveals trade-offs inherent in the Lambda Architecture approach. The increased system complexity, requiring maintenance of both batch and speed processing paths, must be justified by the benefits of comprehensive analytical capabilities. Data quality issues, including missing values, temporal alignment challenges, and API reliability, affect the accuracy and completeness of integrated views. The prototype's limitations, including single-machine deployment and limited data volumes, constrain the evaluation's scope, though conceptual analysis suggests that the architecture would scale effectively with appropriate infrastructure.



The trade-offs between complexity and benefits appear acceptable for Smart City applications where both historical analysis and real-time monitoring are essential. The architecture's ability to provide both perspectives simultaneously, rather than forcing a choice between batch and real-time processing, demonstrates clear value for urban data management scenarios. Future work should address scalability testing at production-scale data volumes, comprehensive user studies with actual city administrators, and quantitative performance benchmarking to provide more rigorous evaluation of the architecture's effectiveness.

7 Conclusion and Future Work

7.1 Summary of Findings

This research successfully designed, implemented, and evaluated a Lambda Architecture prototype [3] that integrates historical traffic flow data with real-time air quality measurements for Ho Chi Minh City. The complete implementation, including source code, configuration files, and documentation, is publicly available in the project repository [18]. The primary contributions demonstrate that the Lambda Architecture's three-layer structure (batch, speed, and serving layers) can be effectively mapped to Smart City data integration requirements [4], enabling both comprehensive historical analysis and real-time responsiveness within a single coherent system.

The implementation validated the practical feasibility of the conceptual architecture, showing that batch and speed processing layers can operate effectively on authentic data sources with realistic volumes and update frequencies. The batch layer successfully processed historical traffic data from the TomTom Traffic API [7] to generate comprehensive views capturing long-term patterns and congestion trends, while the speed layer continuously collected and processed real-time air quality measurements from the OpenAQ API [8], providing low-latency views of current environmental conditions. The serving layer's integration of both views enabled identification of critical periods where traffic congestion and air pollution coincide, demonstrating the practical value of combining historical patterns with real-time measurements.

The evaluation results confirm that the Lambda Architecture provides actionable insights supporting Smart City decision-making, with the integrated system successfully identifying critical periods and revealing recognizable temporal patterns. The architecture's expressiveness is evident in its ability to answer questions requiring both historical analysis and real-time monitoring, showing that the technical integration translates into meaningful analytical capabilities that can inform policy decisions and operational interventions. The research questions have been addressed: RQ1 demonstrated successful integration through temporal alignment and serving layer design; RQ2 identified trade-offs between complexity and comprehensive analytical capabilities; RQ3 confirmed effective critical period identification; and RQ4 provided insights into scalability and deployment considerations for production platforms.

7.2 Limitations

The prototype implementation exhibits several limitations that constrain its direct applicability to production Smart City platforms, as summarized in Table 7.

Table 7: Prototype Limitations

Category	Limitation
Data Coverage	Limited to District 10 over specific time periods; not comprehensive city-wide coverage spanning extended time horizons
Processing Mode	Simple polling rather than true push-based streaming; lacks event-time processing capabilities of production frameworks
Deployment	Single-machine operation with manual configuration; lacks distributed deployment, automated orchestration, and production features
Evaluation Scope	Qualitative assessments and conceptual analysis rather than comprehensive quantitative performance benchmarking at production scale
User Studies	Excludes user studies with actual city administrators; focuses on technical feasibility rather than user acceptance testing

The data range and spatial coverage are limited, with analysis focused on District 10 over specific time periods determined by data availability, rather than comprehensive city-wide coverage spanning extended time horizons. The speed layer employs simple polling rather than true push-based streaming, introducing inherent latency and lacking the event-time processing capabilities of production stream processing frameworks such as Apache Flink [6] or Kafka Streams. The prototype operates on a single machine with manual configuration, lacking distributed deployment, automated orchestration, and production features such as monitoring, alerting, and high availability configurations.

The evaluation scope is constrained by the prototype's scale, with qualitative assessments and conceptual analysis rather than comprehensive quantitative performance benchmarking at production-scale data volumes. The research excludes user studies with actual city administrators, focusing on technical feasibility rather than user acceptance testing or policy impact assessment. Data quality assumptions acknowledge that some missing data and API reliability issues are expected, with the system designed to handle these gracefully rather than requiring complete datasets. The temporal alignment process, while functional, may encounter challenges with larger datasets or more complex temporal relationships, and the spatial assumptions treating measurements as representative of district-wide conditions may be less accurate for larger geographic areas.

7.3 Future Work

Future work should address several directions for extending and improving the Lambda Architecture prototype. Data source extensions could integrate additional IoT sources such as bus GPS data, noise sensors, weather stations, or energy consumption meters, expanding the analytical capabilities to encompass broader Smart City dimensions as defined by Giffinger et al. [4]. Geographic coverage could be extended from District 10 to entire HCMC or multiple districts, enabling city-wide analysis and cross-district comparisons. Longer temporal coverage spanning multiple years would enable analysis of long-term trends, seasonal variations, and the impact of infrastructure changes or policy interventions over extended periods. Technical improvements should focus on migrating from polling-based collection to true streaming frameworks such as Apache Flink [6], Kafka Streams, or Spark Streaming [14], providing event-time processing, state management, and exactly-once semantics essential for production stream processing. Distributed deployment would utilize Spark clusters, Kubernetes [16] orchestration, and cloud infrastructure, enabling horizontal scaling as data volumes grow. Production features including automated monitoring (Prometheus, Grafana), alerting systems, workflow orchestration (Airflow [17]), and high availability configurations would enhance system reliability and operational capabilities.

Advanced analytics capabilities could include machine learning models for forecasting critical periods based on weather conditions, calendar events, or historical patterns, enabling proactive rather than reactive decision-making. Anomaly detection algorithms could identify unusual traffic or pollution events that deviate from expected patterns, supporting early warning systems aligned with public health guidelines [2]. Real-time dashboard development would provide user-friendly interfaces for city administrators, while API development would enable programmatic access to integrated views for custom applications. Comparative studies evaluating Lambda Architecture against alternative approaches such as Kappa Architecture or hybrid solutions would provide additional insights into architectural trade-offs for Smart City data platforms, contributing to the broader understanding of scalable real-time data systems [3].

References

- [1] United Nations, Department of Economic and Social Affairs, Population Division, “World urbanization prospects: The 2018 revision,” United Nations, New York, USA, Tech. Rep., 2018. [Online]. Available: <https://population.un.org/wup/>
- [2] World Health Organization, “Who global air quality guidelines: particulate matter (pm2.5 and pm10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide,” World Health Organization, Geneva, Switzerland, Tech. Rep., 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240034228>
- [3] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable real-time data systems*. Manning Publications, 2015.
- [4] R. Giffinger, C. Fertner, H. Kramar, R. Kalasek, N. Pichler-Milanović, and E. Meijers, “Smart cities ranking: An effective instrument for the positioning of cities?” *ACE: Architecture, City and Environment*, vol. 2, no. 5, pp. 7–25, 2007.
- [5] Apache Software Foundation, “Apache hadoop,” 2025, accessed: 2025. [Online]. Available: <https://hadoop.apache.org/>
- [6] —, “Apache flink,” 2025, accessed: 2025. [Online]. Available: <https://flink.apache.org/>
- [7] TomTom, “Tomtom traffic api,” 2025, accessed: 2025. [Online]. Available: <https://developer.tomtom.com/traffic-api>
- [8] OpenAQ, “Openaq api,” 2025, accessed: 2025. [Online]. Available: <https://openaq.org/>
- [9] The pandas development team, “pandas: Python data analysis library,” 2025, accessed: 2025. [Online]. Available: <https://pandas.pydata.org/>
- [10] NumPy Developers, “Numpy,” 2025, accessed: 2025. [Online]. Available: <https://numpy.org/>
- [11] Apache Software Foundation, “Apache kafka,” 2025, accessed: 2025. [Online]. Available: <https://kafka.apache.org/>
- [12] Delta Lake Project, “Delta lake,” 2025, accessed: 2025. [Online]. Available: <https://delta.io/>



- [13] MinIO, Inc., “Minio,” 2025, accessed: 2025. [Online]. Available: <https://min.io/>
- [14] Apache Software Foundation, “Apache spark,” 2025, accessed: 2025. [Online]. Available: <https://spark.apache.org/>
- [15] Python Software Foundation, “Python programming language,” 2025, accessed: 2025. [Online]. Available: <https://www.python.org/>
- [16] The Kubernetes Authors, “Kubernetes,” 2025, accessed: 2025. [Online]. Available: <https://kubernetes.io/>
- [17] Apache Software Foundation, “Apache airflow,” 2025, accessed: 2025. [Online]. Available: <https://airflow.apache.org/>
- [18] XDcobra, “Hcmc air traffic kafka monitor: Lambda architecture for real-time traffic speed and air quality data,” 2025, gitHub repository. [Online]. Available: <https://github.com/XDcobra/hcmc-air-traffic-kafka-monitor>